

《软件工程理论基础》课程实验二

实验说明与提交要求

本次实验围绕两种程序形式化验证工具展开：

- **Frama-C/WP**：ANSI/ISO C Specification Language（ACSL）是一种针对 C 语言程序源码的形式化规范注释语言，能够描述程序函数的前置条件、后置条件、不变式、断言等性质。Frama-C 是一个用于 C 语言程序静态分析的开源平台，其 WP 插件支持静态验证 ACSL 规约与目标程序的一致性。若你从未使用过该工具，可遵循[官方网站下载指南](#)所示的步骤安装。
- **Verus**：Verus 是一种针对 Rust 程序的形式化验证工具与规约语言，通过 SMT 求解器（Z3）自动验证程序实现是否满足规约。Verus 以 Rust 宏（`verus!`）的形式融合规约与实现代码，支持验证包括线性算术、数组操作、数据结构不变式等多种性质。若你从未使用过该工具，可遵循[Verus 官方安装指南](#)所示的步骤进行安装。

在本实验中，你需要安装这两种工具，熟悉其基本用法，并尝试为几个简单程序编写规约以完成验证任务。

本次实验最终需要你提交一个名为 `学号_姓名.zip` 的压缩包，其使用 `unzip` 解压后应形成如下文件结构：

```
学号_姓名/  
├─ foo1.c  
├─ foo2.rs  
├─ search.rs  
├─ sort.c  
└─ 学号_姓名.pdf
```

其中，`学号_姓名.pdf` 为一份项目报告，其中请简要回答下文任务中所涉及的问题并阐述你解决问题的思路；其余文件的内容详见下文的具体任务要求。文件名中的各处 `学号` 请替换为你自己的学号，`姓名` 请填写你自己的姓名。请将压缩包提交至 <https://box.nju.edu.cn/u/d/1c7b943d9a54462a897c/>，提交截止时间为**2026年6月19日23点59分**。

第一部分：Frama-C/WP

任务1.1：简单数值程序的规约编写与验证

考察下列 C 语言函数。

```
/*@  
    requires 0 < n <= 100;  
*/  
int foo1(int n) {  
    int i = 0, x = 1, y = 0;  
  
    while(i < n) {  
        if(i % 2 == 0)  
            x = 2;  
        else  
            x = 1;  
        y += x;  
        i++;  
    }  
    //@ assert fn: x == ?; // TODO  
    //@ assert gn: y == ?; // TODO  
    return y;  
}
```

完成以下任务：

1. 该函数执行至 `return y;` 语句时，存在函数 f, g 使得变量 x, y 的值满足 $x = f(n), y = g(n)$ 。请给出这样的函数 f 与 g 。
2. 将你给出的函数 f 与 g 分别填入代码中的 `assert` 断言中，并补全必要的 ACSL 规约，使得 Frama-C/WP 工具（由命令 `frama-c -wp [SourcePath] -then -report` 调用）能够证明其完全正确性。请将完整规约与源码整体写入 `foo1.c` 文件，置于压缩包下。

任务1.2：排序算法正确性验证

考察下列 C 语言函数：

```
void bubble_sort(int arr[], int n) {  
    for (int i = 0; i < n - 1; i++) {  
        for (int j = 0; j < n - 1 - i; j++) {  
            if (arr[j] > arr[j + 1]) {  
                int t = arr[j];  
                arr[j] = arr[j + 1];  
                arr[j + 1] = t;  
            }  
        }  
    }  
}
```

试为其编写完整 ACSL 规约，并使用 Frama-C/WP 工具证明：经过该函数处理后的数组 `arr` 一定是有序的。请将完整规约与源码整体写入 `sort.c` 文件，置于压缩包下。

第二部分：Verus

任务2.1：简单数值程序的规约编写与验证

考察下列 Rust 函数。

```
use vstd::prelude::*;  
  
verus! {  
  
fn foo1(n: u32) -> (ret: (u32, u32))  
    requires  
        0 < n <= 100,  
    ensures  
        // TODO: 描述 ret.0, ret.1 与 n 之间的关系  
{  
    let mut i: u32 = 0;  
    let mut a: u32 = 3;  
    let mut b: u32 = 1;  
  
    while i < n  
        invariant  
            // TODO: 补全循环不变式  
    {  
        a = a + 5u32;  
        b = b + 2u32;  
        i = i + 1;  
    }  
    (a, b)  
}
```

试将该函数返回值与输入参数 `n` 的关系填入 `ensures` 语句中（用 `ret.0` 与 `ret.1` 可引用返回值），并补全循环不变式，使得 Verus 工具（由 `verus --crate-type=lib [SourcePath]` 命令调用）能够证明其完全正确性。请将完整规约与源码整体写入 `foo2.rs` 文件，置于压缩包下。

任务2.2：二分查找算法正确性验证

考察下列 Rust 函数：

```
use vstd::prelude::*;  
  
verus! {  
  
fn binary_search(arr: &[i32], n: usize, target: i32) -> (ret: (bool, usize))  
    requires  
        n == arr.len(),  
        n >= 1,  
        // 数组已按升序排列  
        forall|a: int, b: int|  
            0 <= a && a <= b && b < n ==> #[trigger] arr[a] <= #[trigger] arr[b],  
    ensures  
        // TODO: 描述 ret.0 为 true/false 时分别满足什么性质  
{  
    let mut lo: usize = 0;  
    let mut hi: usize = n - 1;  
    while lo != hi  
        invariant  
            // TODO: 补全循环不变式  
    {  
        let mid = lo + (hi - lo) / 2;  
        if arr[mid] < target {  
            lo = mid + 1;  
        } else if arr[mid] > target {  
            hi = mid;  
        } else {  
            return (true, mid);  
        }  
    }  
    if arr[lo] == target {  
        (true, lo)  
    } else {  
        (false, 0)  
    }  
}
```

试为其编写完整 Verus 规约，并使用 Verus 工具证明该二分查找实现的正确性。具体而言，`ensures` 中至少应描述：

- 若 `ret.0` 为真，则 `ret.1` 是有效的下标且 `arr[ret.1 as int] == target`；
- 若 `ret.0` 为假，则 `target` 不在数组中。

请将完整规约与源码整体写入 `search.rs` 文件，置于压缩包下。

提示：循环不变式中需刻画 `lo` 与 `hi` 的边界关系，以及 `target` 如果存在于数组中则其下标必然落在 `[lo, hi]` 区间内；有序性（`arr` 升序）也需要置于不变式中以便推导。

第三部分（可选）

在该部分中，你需要考察一系列插装有 ACSL 规约的 C 语言程序片段，尝试对程序与规约进行理解，并回答问题。

- 你需要考察总共 70 个程序片段，并针对每个片段回答两个问题，预计完成用时约 30~50 分钟。
- 提交有效结果后，该次实验总分（满分 100）视为额外增加 10 分（可超出满分）。
- 问卷地址：<http://172.27.143.102:8000/>（需在校园网环境下访问）